

گزارش پروژه پردازش زبان طبیعی

مهربد اعتمادی(1400012268036) – هانیه وطنی(1400012268035)

الف) روی مجموعه داده PQAD مدل ParsBERT را Fine-Tune کنید. نتایج معیارهای F1-Score و Exact Match (EM) گزارش دهید.

در ابتدای کار نام کامل دیتاست را از سایت برداشته و آن را لود میکنیم.

```
#Question 1
!pip install transformers datasets accelerate torch
from datasets import load_dataset
path = "Gholamreza/pquad"
dataset = load_dataset(path)
```

برای لود کردن مدل از پیش آموزش داده شده هم توکنایزر و هم خود مدل را لود میکنیم. همچنین کتابخانه های مربوط به پروژه را ایمپورت میکنیم.

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForQuestionAnswering, TrainingArguments, Trainer

# loading parsbert and tokenizer
model_name = "HooshvareLab/bert-base-parsbert-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForQuestionAnswering.from_pretrained(model_name)
```

داده های آموزش، تست و اعتبارسنجی را جدا کرده و داده ها(توکن) خالی را از آن خارج میکنیم.

```
#export empty data
train_data = dataset['train'].filter(lambda x:x['answers']['text'])
val_data = dataset['validation'].filter(lambda x:x['answers']['text'])
test_data = dataset['test'].filter(lambda x:x['answers']['text'])
```

سپس پیش پردازش های لازم را بر روی ورودی انجام میدهم. دو نمونه از آنها را در ادامه شرح میدهم.

```
# Tokenize the questions and contexts
inputs = tokenizer(
    examples["question"], # List of questions
    examples["context"], # List of contexts
    truncation=True, # Truncate sequences longer than max_length
    padding="max_length", # Pad sequences to max_length
    return_offsets_mapping=True, # Return character offsets for each token
    max_length=512, # Maximum sequence length
    stride=128 # Stride for overlapping chunks
)
```

این تابع توکنایزر دو لیست سوال و کانتکست را میگیرد و ورودی را توکن بندی میکند، پارامترهایی برای آن تعریف کردیم تا دقت بالاتر برود، برای مثال ماکسیمم طول دنباله ای 512 میباشد یا داده ای که طول آن از ماکسیمم بیشتر باشد کوتاه میشود.

```
# Add start and end positions to the inputs dictionary
inputs["start_positions"] = start_positions
inputs["end_positions"] = end_positions
```

پیش پردازش دیگری که میتوان انجام داد پیدا کردن نقطه شروع و پایان توکن ها میباشد که به دیکشنری ورودی اضافه با کلیدهای مربوطه اضافه میشود.

```
# Apply preprocessing to the training, validation, and test datasets
tokenized_train = train_data.map(preprocess, batched=True, remove_columns=train_data.column_names)
tokenized_val = val_data.map(preprocess, batched=True, remove_columns=val_data.column_names)
tokenized_test = test_data.map(preprocess, batched=True, remove_columns=test_data.column_names)
```

Map: 100%		48273/48273 [00:51<00:00, 990.37 examples/s]
Map: 100%		5995/5995 [00:07<00:00, 928.50 examples/s]
Map: 100%		6088/6088 [00:06<00:00, 1060.81 examples/s]

در نهایت پیش پردازش را روی تمام نوع داده ها اعمال میکنیم و همونطور که در تصویر میبینیم به شکل کاملی این پروسه طی شده است.

```
# Define training arguments for fine-tuning the model
training_args = TrainingArguments(
    output_dir="./qa-finetuned", # Directory to save the model and outputs
    evaluation_strategy="epoch", # Evaluate the model at the end of each epoch
    learning_rate=3e-5, # Learning rate for optimization
    per_device_train_batch_size=8, # Batch size per device during training
    per_device_eval_batch_size=8, # Batch size per device during evaluation
    num_train_epochs=2, # Number of training epochs
    weight_decay=0.01, # Weight decay for regularization
    logging_dir="./logs", # Directory to store logs
    save_strategy="epoch", # Save the model at the end of each epoch
    fp16=True, # Enable mixed precision training (FP16) for faster training on GPUs
)

# Initialize the Trainer for fine-tuning
trainer = Trainer(
    model=model, # The pre-trained model to fine-tune
    args=training_args, # Training arguments defined above
    train_dataset=tokenized_train, # Preprocessed training dataset
    eval_dataset=tokenized_val, # Preprocessed validation dataset
    tokenizer=tokenizer # Tokenizer used for preprocessing
)
```

قبل از انجام ترین آجکت ترینر و همچنین آرگومان های آموزش را مشخص میکنیم. برای مثال اندازه بچ های داده را برای آموزش و ارزیابی برابر با 8 میگیریم، عملیات آموزش را در دو اپوک انجام میدهم و در پایان هر اپوک داده ها و نتایج ارزیابی میشوند.

برای پارامترهای آجکت ترینر نیز مدل، توکنایزر و دیتاهای آموزش و آرگومان ها را به آن میدهم تا در مرحله بعدی با این ویژگی های مشخص شده مدل را آموزش دهیم.

```
trainer.train()

wandb: WARNING The `run_name` is currently set to the same value as `TrainingArguments.output_dir`. If this was not i
wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: You can find your API key in your browser here: https://wandb.ai/authorize
wandb: Paste an API key from your profile and hit enter, or press ctrl+c to quit: wandb: ERROR API key must be 40 char
wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: You can find your API key in your browser here: https://wandb.ai/authorize
wandb: Paste an API key from your profile and hit enter, or press ctrl+c to quit: wandb: Appending key for api.wandb.a
wandb: Using wandb-core as the SDK backend. Please refer to https://wandb.me/wandb-core for more information.
Tracking run with wandb version 0.19.4
Run data is saved locally in /content/wandb/run-20250125_201633-vtlp1jzi
Syncing run ./qa-finetuned to Weights & Biases (docs)
View project at https://wandb.ai/mehrbodetemadi82-university-of-guilan/huggingface
View run at https://wandb.ai/mehrbodetemadi82-university-of-guilan/huggingface/runs/vtlp1jzi
[12136/16000 42:39 < 13:35, 4.74 it/s, Epoch 1.52/2]

Epoch Training Loss Validation Loss
1 3.667700 3.635697

[16000/16000 57:03, Epoch 2/2]

Epoch Training Loss Validation Loss
1 3.667700 3.635697
2 3.555600 3.706312
```

پس از انجام مراحل بالا مدل را با تابعی که در تصویر میبینیم آموزش میدهم، همانطور که میبینیم در پایان هر اپوک مقدار LOSS ترینر و اعتبارسنجی مشخص شده است. برای آموزش این مدل و این دیتاست نیاز به یک کد داریم که با کلیک بر لینک بالا و ثبت نام میتوانیم آن را دریافت کنیم. در ادامه از مدل آموزش دیده شده تست پیشبینی گرفته و آن را ارزیابی میکنیم.

```
# Extract questions and contexts from the batch
questions = [item['question'] for item in batch]
contexts = [item['context'] for item in batch]
```

در این تابع بچ دیتا که شامل پرسش و زمینه آن میباشد به عنوان ورودی داده میشود و سپس پرسش ها و زمینه های آن در دو لیست جدا نگه داری میشوند.

در ادامه مدل با تابع زیر با پارامترهای مدل فاین تون شده، توکنایزر، دیتاست و سائز بچ مساوی 16 نتیجه را پیشبینی میکند، سائز بچ را به این دلیل زیادتر گرفتیم چون زمان بسیار زیادی برای پیشبینی صرف میشد.

```
def predict_batch(model, tokenizer, dataset, batch_size=16):
    predictions = [] # List to store predicted answers
    model.eval() # Set the model to evaluation mode

    # Create a DataLoader with a custom collate function
    dataloader = DataLoader(dataset, batch_size=batch_size, collate_fn=collate_fn)

    # Iterate over batches in the DataLoader
    for batch in dataloader:
        # Move input tensors to the model's device (GPU or CPU)
        inputs = {key: value.to(model.device) for key, value in batch.items()}

        # Perform inference (no gradient calculation)
        with torch.no_grad():
            outputs = model(**inputs)

        # Find the start and end token indices for each answer
        start_indices = torch.argmax(outputs.start_logits, dim=1) # Start positions
        end_indices = torch.argmax(outputs.end_logits, dim=1) # End positions
```

```
# Extract answers for each sample in the batch
for i in range(len(batch['input_ids'])):
    try:
        # Get the start and end token indices for the answer
        start_index = start_indices[i].item()
        end_index = end_indices[i].item()

        # Extract the answer tokens and decode them into text
        answer_tokens = batch["input_ids"][i][start_index:end_index + 1]
        answer = tokenizer.decode(answer_tokens, skip_special_tokens=True)

        # Add the predicted answer to the list
        predictions.append(answer)
    except IndexError:
        # If there's an issue with prediction, add an empty string
        predictions.append("")

return predictions
```

این تابع از دو تابع فور تشکیل شده است، تابع فور بیرونی هر بچ را انتخاب میکند و تابع فور داخلی به ازای هر بچ داده سمپل را استخراج و پیشبینی مربوط به آن را انجام میدهد. در انتها پاسخی که پیشبینی شده اند به لیست افزوده و برگردانده میشوند.

```
# Perform predictions on the test and validation datasets
predictions_test = predict_batch(model, tokenizer, test_data, batch_size=16)
predictions_val = predict_batch(model, tokenizer, val_data, batch_size=16)
```

عمل پیشبینی را هم با داده تست و هم داده اعتبارسنجی انجام میدهیم.

```
results = evaluate(val_data, predictions_val)
print(f"em on val: {results['exact_match']}")
print(f"f1 on test: {results['f1']}")

results = evaluate(test_data, predictions_test)
print(f"em on val: {results['exact_match']}")
print(f"f1 on test: {results['f1']}")
```

در انتها با استفاده از `Metric = load("squad")` که یک معیار ارزیابی است خروجی های خواسته شده در سوال را چاپ میکنیم.

```
em on val: 0.0
f1 on test: 4.155367290970469
em on val: 0.0
f1 on test: 3.620822352426151
```

همچنین لازم به ذکر است پس از آموزش مدل آن را با دستورات زیر و هر موقع که نیاز داشتیم آن را لود میکنیم.

```
#save model
from google.colab import drive
drive.mount('/content/drive')
save_path = "/content/drive/MyDrive/my_finetuned_model"
model.save_pretrained(save_path)
tokenizer.save_pretrained(save_path)
```

```
#load model
from google.colab import drive
drive.mount('/content/drive')
model = AutoModelForQuestionAnswering.from_pretrained('/content/drive/MyDrive/my_finetuned_model')
tokenizer = AutoTokenizer.from_pretrained('/content/drive/MyDrive/my_finetuned_model')
```

ب) از طریق روش های Zero-Shot و Few-Shot مدل Llama-3.1-8B-4bit را برای پرسش و پاسخ استفاده کنید. نتایج را بر اساس معیارهای F1-Score و Exact Match (EM) تحلیل و مقایسه کنید.

در ابتدا لازم است به تفاوت Zero shot و Few shot بپردازیم.

در زیروبات مدل بدن هیچ مثال آموزشی بر اساس دستورالعمل پاسخ را تولید میکند.

Prompt = “what is the capital of Iran”?

در فیووبات مدل با چند مثال الگوی پاسخدهی را یادمیگیرد.

Prompt=“ Q: what is the capital of France? A: It is Paris!

Q: what is the capital of Germany? A: “

در ادامه به بررسی کد میپردازیم.

```
MODEL_NAME = "unsloth/llama-3-8b-bnb-4bit"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForCausalLM.from_pretrained(MODEL_NAME)
generator = pipeline("text-generation", model=model, tokenizer=tokenizer)
```

پس از لود کردن دیتاست مدل گفته شده را لود میکنیم.

در ابتدا چند داده نمونه مینویسیم تا کارکرد این دو روش رو متوجه بشیم و سپس به دو روش مقادیر خواسته شده در سوال را به دست میاوریم.

```

# Initialize a list of questions
questions0 = [
    "Where is the capital of France?",
    "Where is the capital of Germany?"
]

# Respond to each question
for question in questions0:
    # Tokenize the question
    inputs = tokenizer(question, return_tensors="pt", truncation=True).to(device)

    # Generate an answer using the model
    with torch.no_grad(): # Disable gradient calculation for inference
        outputs = model.generate(
            *inputs, # Pass tokenized inputs to the model
            max_length=100, # Maximum length of the generated text
            temperature=0.7, # Controls randomness (lower = more deterministic)
            top_k=50, # Limits sampling to the top-k tokens
            top_p=0.9, # Nucleus sampling: limits sampling to the top-p probability mass
            no_repeat_ngram_size=2 # Prevents repetition of n-grams of size 2
        )

    # Decode the generated tokens into text
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    # Print the question and generated answer
    print(f"Question: {question}\nAnswer: {answer}\n")

```

در اینجا یک لیست سوالات به مدل داده میشود و مدل قرار است به هر سوال جواب بدهد. در ابتدا سوالات توکن بندی شده و سپس به وسیله مدل یک پاسخ به آنها داده تولید میشود. در انتها توکن هایی که تولید شدند به تکست تبدیل شده و خروجی مشخص میشود. این روش زیروشات است.

```

# Define a few examples of questions and answers for few-shot learning
examples = [
    {"question": "Where is the capital of Italy?", "answer": "The capital of Italy is Rome."},
    {"question": "Where is the capital of Japan?", "answer": "The capital of Japan is Tokyo."},
]

# Define new questions to ask the model
questions_few = [
    "Where is the capital of Russia?",
    "Where is the capital of Germany?",
    "Where is the capital of Brazil?"
]

```

اما در روش فیوشات پروسه ای شبیه اما با کمی تفاوت طی میشود. در ابتدا سوالات و جواب های نمونه را میدهم سپس سوالات را مطرح میکنیم و هر دو لیست را به ورودی تابع میدهم.

خروجی به شکل زیر است:

```
Question: Where is the capital of Russia?
Answer: Moscow is capital city of Russian Federation.

Question: Where is the capital of Germany?
Answer: Berlin is Germany's capital.

Question: Where is the capital of Brazil?
Answer: Brazil's capital is Brasilia.
```

در ادامه به بررسی معیارها بر روی دیتاست انتخابی می‌رسیم.

```
[ ] def process_dataset(dataset, split="train"):

    processed = [] # List to store processed data

    # Iterate over each item in the specified dataset split
    for item in dataset[split]:
        # Check if the required keys exist and the answer is non-empty
        if not ("answers" in item and "text" in item["answers"] and len(item["answers"]["text"]) > 0):
            continue # Skip invalid items

        # Append the processed item to the list
        processed.append({
            "question": item["question"], # Extract the question
            "reference": item["answers"]["text"][0] # Extract the first reference answer
        })

    return processed
```

ابتدا آیتم‌ها رو پیش پردازش می‌کنیم، برای مثال چک می‌کنیم کلید مورد نیاز وجود دارد و پاسخ خالی است دیتاست رو بر اساس نوع خاصی از داده که در اینجا ترین است اسپلین می‌کنیم.

```
def exact_match(pred, ref):
    """
    Calculate the Exact Match (EM) score between a predicted answer and a reference answer.

    Args:
        pred (str): The predicted answer.
        ref (str): The reference (correct) answer.

    Returns:
        int: 1 if the answers match exactly, otherwise 0.
    """
    return 1 if pred.strip() == ref.strip() else 0
```

تابع EM را این گونه می‌نویسیم: اگر پیش‌بینی با رفرنس (واقعیت) یکسان باشد یک برمی‌گرداند.


```

# Tokenize the predicted and reference answers
pred_tokens = pred.strip().split()
ref_tokens = ref.strip().split()

# Find common tokens between the predicted and reference answers
common_tokens = set(pred_tokens) & set(ref_tokens)
if len(common_tokens) == 0:
    return 0 # No overlap, F1 is 0

# Calculate precision and recall
precision = len(common_tokens) / len(pred_tokens)
recall = len(common_tokens) / len(ref_tokens)

# Calculate F1 score
return 2 * (precision * recall) / (precision + recall) if (precision + recall) != 0 else 0

```

تابع F1 به این صورت است که پاسخ های واقعی و پیشبینی را توکن بندی کرده و بررسی میکند چقدر از داده های آنها همپوشانی دارد، اگر نداشته باشد صفر و اگر داشته باشد مقدار آن را بر اساس فرمول برمیگرداند.

```

# Construct the prompt based on the method
if method == "zero-shot":
    prompt = f"Question: {question}\nAnswer:"
elif method == "few-shot":
    # Use 2 examples from the dataset as few-shot examples
    examples = "\n".join(
        [f"Question: {sample_data[i]['question']}\nAnswer: {sample_data[i]['reference']}"
         for i in range(2)]
    )
    prompt = f"{examples}\nQuestion: {question}\nAnswer:"

```

در بخش ارزیابی بر اساس اینکه روش چی باشد پرامپ ساخته میشود. اگر زیروشات باشد فقط سوال را وارد میکنیم و اگر فیوشات باشد به عنوان نمونه دو مثال از سوال و جواب در دیتاست انتخاب میشود تا مدل بر اساس آنها به سوال پاسخ بدهد.

```
# Generate the answer using the model
response = generator(
    prompt,
    max_new_tokens=50, # Maximum number of tokens to generate
    truncation=True,   # Truncate the input if it exceeds the model's max length
    num_return_sequences=1, # Generate only one sequence
    pad_token_id=tokenizer.eos_token_id, # Use the EOS token for padding
    return_full_text=False, # Do not include the prompt in the output
    batch_size=2 # Process in batches of 2
)[0]['generated_text']

# Extract the predicted answer
try:
    pred_answer = response.split("Answer: ")[-1].split("\n")[0].strip()
except:
    pred_answer = "" # Handle cases where extraction fails
```

در نهایت مدل بر اساس پرامپت ورودی و پارامترهایی مانند ماکسیمم طول توکن برای تولید متن را تولید می‌کند و مقادیر پیش‌بینی شده از لیست پاسخ‌های پیش‌بینی شده استخراج می‌شوند و به تابع مربوط به محاسبه معیارها پاس داده شده و مقادیر آن چاپ می‌شود.

```
Zero-Shot: {'EM': '0.00%', 'F1': '4.69%', 'Total Samples': 50}
Few-Shot: {'EM': '2.00%', 'F1': '7.02%', 'Total Samples': 50}
```

در این پروژه روش دیگری را برای کار کردن پیش بردیم برای بهبود نتایج که در ادامه به شرح آن می‌پردازیم. در اینجا ابتدا تکست را نرمالایز می‌کنیم و سپس به محاسبه معیارها می‌پردازیم.

```
def normalize_text(text):
    """
    Normalize text by:
    1. Stripping leading/trailing whitespace.
    2. Removing all spaces and zero-width non-joiner characters.
    3. Converting the text to lowercase.

    Args:
        text (str): The input text to normalize.

    Returns:
        str: The normalized text.
    """
    return text.strip().replace(" ", "").replace("\n", "").lower()
```

برای تولید تکست از تابع پایپ لاین بهره میبریم.

```
# Initialize a text-generation pipeline
generator = pipeline(
    "text-generation", # Task type: text generation
    model=model,        # The pre-trained model to use
    tokenizer=tokenizer, # The tokenizer associated with the model
    max_new_tokens=30,   # Maximum number of tokens to generate
    temperature=0.3,     # Controls randomness (lower = more deterministic)
    top_p=0.9,           # Nucleus sampling: limits sampling to the top-p probability mass
    do_sample=False,     # Disable sampling (use greedy decoding)
    repetition_penalty=1.2, # Penalize repeated tokens to reduce redundancy
    batch_size=4,        # Process inputs in batches of 4
    truncation=True,     # Truncate inputs longer than the model's maximum length
)
```

در این تابع برای پردازش بهتر ماکسیمم تعداد توکن ها رو 30 میکنیم همچنین برای رندوم بودن و تکراری بودن توکن حد و حدود میگذاریم.

تابع ارزیابی نیز در اینجا مانند حالت قبلی کار میکند.

```
Zero-Shot: {'EM': '0.00%', 'F1': '0.00%', 'Samples': 50}
Few-Shot: {'EM': '4.00%', 'F1': '4.00%', 'Samples': 50}
```

اما با اعمال این روش تغییری در وضعیت معیارها به وجود نیامد. در کل میتوان گفت مقدار درصد این معیارها در فیوشات به دلیل داشتن زمینه و داده نمونه بیشتر از زیروشات میباشد.

چالش مهم و اساسی در دو سوال گذشته به دست آوردن دقت بالا در تست و آموزش برای مدل بود. به نظر میاید اهمیت پیش پردازش درست و اصولی برای آن که مدل به خوبی بتواند با دیتاست فیت شود از اهمیت بالایی برخوردار است. همچنین چالش دیگری که باهاش مواجه هستیم شناخت درست و دقیق از مدل و معماری آن میباشد، پس برای آن که بتوانیم خروجی بهتری ز این معیارها داشته باشیم اعمال پیش پردازش مناسب بر دیتاست و همچنین شناختن معماری و نحوه عملکرد مدل نکته هایی بسیار اساسی و مهم هستند.

ج) روی مجموعه داده PQuAD مدل Llama-3.1-8B-4bit را با روش هایی نظیر LoRA و QLoRA Fine-Tune کنید. در خروجی پروژه یک مدل آموزش دیده و بهینه سازی شده که قادر به پاسخ دهی به سوالات فارسی باشد مورد انتظار است. و در معیار هایی نظیر F1-Score و Exact Match (EM) دقت بالایی داشته باشد.

این قطعه کد مجموعه داده ی PQuAD (نسخه ی فارسی SQuAD) را از هگینگ فیس بارگیری می کند. این مجموعه داده برای آموزش و ارزیابی مدل های پاسخ به سوالات (QA) استفاده می شود.

```
[ ] from datasets import load_dataset
    path = "Gholamreza/pquad"
    dataset = load_dataset(path)
```

این قطعه کد یک مدل زبانی بزرگ (مانند LLaMA-3) را با پیکربندی کوانتیزاسیون ۴ بیتی بارگیری می کند تا مصرف حافظه را کاهش دهد. مدل و توکنایزر از Hugging Face بارگیری شده و روی دستگاه های موجود (مانند GPU) به طور خودکار قرار می گیرند. در نهایت، مدل و توکنایزر چاپ می شوند تا از صحت بارگیری اطمینان حاصل شود.

```
] import torch # Import PyTorch
   from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

   # Configure 4-bit quantization
   quantization_config = BitsAndBytesConfig(
       load_in_4bit=True, # Load the model in 4-bit mode
       bnb_4bit_use_double_quant=True, # Use double quantization
       bnb_4bit_quant_type="nf4", # Quantization type
       bnb_4bit_compute_dtype=torch.float16 # Compute dtype
   )

   # Load the pre-trained model and tokenizer
   model_name = "unsloth/llama-3-8b-bnb-4bit"
   model = AutoModelForCausalLM.from_pretrained(
       model_name,
       quantization_config=quantization_config, # Pass the quantization config
       device_map="auto" # Automatically map the model to available devices
   )
   tokenizer = AutoTokenizer.from_pretrained(model_name)

   # Check if the model and tokenizer are loaded correctly
   print(model)
   print(tokenizer)
```

این قطعه کد از LoRA برای تنظیم دقیق مدل زبانی با پارامترهای کمتر استفاده می‌کند. LoRA به لایه‌های خاصی (`v\_proj` و `q\_proj`) اعمال می‌شود و مدل برای کارهای زبانی بهینه می‌شود. در نهایت، مدل با لایه‌های LoRA چاپ می‌شود.

```
[ ] from transformers import DynamicCache # Use DynamicCache instead of Cache
    from peft import LoraConfig, get_peft_model

    # Configure LoRA
    lora_config = LoraConfig(
        r=8, # Rank of the low-rank matrices
        lora_alpha=32, # Scaling factor
        target_modules=["q_proj", "v_proj"], # Target modules to apply LoRA
        lora_dropout=0.1, # Dropout rate
        bias="none", # Bias type
        task_type="CAUSAL_LM" # Task type
    )

    # Apply LoRA to the model
    model = get_peft_model(model, lora_config)

    # Print the model to see the LoRA layers
    print(model)
```

این قطعه کد داده‌ها را برای آموزش مدل آماده می‌کند: سوالات و متن‌ها را تمیز کرده، ترکیب می‌کند، توکن‌سازی می‌کند و برچسب‌ها (پاسخ‌ها) را تنظیم می‌کند، سپس این پیش‌پردازش را روی کل مجموعه داده اعمال می‌کند تا داده‌ها آماده آموزش شوند.

```
[ ] def preprocess_function(examples):
    questions = [q.strip() for q in examples["question"]]
    contexts = [c.strip() for c in examples["context"]]
    answers = []

    for a in examples["answers"]:
        if "text" in a and len(a["text"]) > 0: # Check if "text" exists and is not empty
            answers.append(a["text"][0])
        else:
            answers.append("") # Use an empty string as a default value

    # Combine question and context
    inputs = [f"question: {q} context: {c}" for q, c in zip(questions, contexts)]

    # Tokenize the inputs and answers
    model_inputs = tokenizer(inputs, max_length=512, truncation=True, padding="max_length")
    labels = tokenizer(answers, max_length=512, truncation=True, padding="max_length")

    # Set the labels
    model_inputs["labels"] = labels["input_ids"]

    return model_inputs

# Apply the preprocessing function to the dataset
tokenized_datasets = dataset.map(preprocess_function, batched=True)
```

این قطعه کد تعداد نمونه‌های موجود در هر بخش (train, validation, test) از مجموعه داده را چاپ می‌کند.

```
[ ] # Check the number of examples in each split
    for split in dataset:
        print(f"{split}: {len(dataset[split])} examples")
```

```
train: 63994 examples
validation: 7976 examples
test: 8002 examples
```

این قطعه کد تنظیمات آموزش مدل را تعریف می‌کند، شامل نرخ یادگیری، اندازه بچ، تعداد دوره‌ها، وزن‌دهی و فعال‌سازی آموزش با دقت مختلط (FP16) برای بهینه‌سازی عملکرد.

```
[ ] from transformers import TrainingArguments

# Define training arguments with a smaller batch size
training_args = TrainingArguments(
    output_dir="./results",
    evaluation_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=2, # Reduce batch size
    per_device_eval_batch_size=8, # Increase evaluation batch size
    num_train_epochs=3,
    weight_decay=0.01,
    save_total_limit=2,
    fp16=True, # Enable mixed precision training
    logging_dir="./logs",
    logging_steps=10,
)
```

این قطعه کد روی داده‌های آموزش از مجموعه‌ی توکن‌شده تکرار می‌کند و لیست‌ها را به تانسورهای PyTorch تبدیل می‌کند تا برای آموزش مدل آماده شوند.

```
[ ] import torch

#iterate over the training split of the tokenized dataset
for batch in tokenized_datasets["train"]:
    # Convert lists to tensors
    batch["input_ids"] = torch.tensor(batch["input_ids"])
    batch["attention_mask"] = torch.tensor(batch["attention_mask"])
    batch["labels"] = torch.tensor(batch["labels"])
```

این قطعه کد از `PartialState` در کتابخانه‌ی `accelerate` استفاده می‌کند تا وضعیت پردازش‌ها را مدیریت کند. اگر پردازش فعلی، پردازش اصلی باشد، پیام "This is the main process" چاپ می‌شود.

```
from accelerate import PartialState

# Initialize PartialState
state = PartialState()

# Use the state in your code
if state.is_main_process:
    print("This is the main process")
```

این قطعه کد یک شیء `Trainer` از Hugging Face ایجاد می‌کند که برای آموزش مدل استفاده می‌شود. مدل، تنظیمات آموزش، داده‌های آموزش و ارزیابی، و توکنایزر به عنوان ورودی به آن داده می‌شوند.

```
from transformers import Trainer

# Initialize the Trainer without the 'accelerator' argument
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    tokenizer=tokenizer,
)
```

این قطعه کد ۱۰۰ نمونه از داده‌های ارزیابی را انتخاب کرده و مدل را روی آن‌ها ارزیابی می‌کند، سپس نتایج ارزیابی را چاپ می‌کند.

```
small_eval_dataset = tokenized_datasets["validation"].select(range(100)) # Use only 100 samples
eval_results = trainer.evaluate(small_eval_dataset)

# Print evaluation results
print(f"Evaluation results: {eval_results}")
```

این قطعه کد Google Drive را به محیط Google Colab متصل می‌کند و مدل و توکنایزر آموزش‌دیده را در مسیر مشخص‌شده داخل Google Drive ذخیره می‌کند.

```
from google.colab import drive

# Mount Google Drive
drive.mount('/content/drive')

# Define the path in Google Drive where you want to save the model
save_path = "/content/drive/MyDrive/fine-tuned-llama"

# Save the fine-tuned model and tokenizer
model.save_pretrained(save_path)
tokenizer.save_pretrained(save_path)

print(f"Model and tokenizer saved to {save_path}")
```

این قطعه کد مدل و توکنایزر آموزش‌داده‌شده را از Google Drive بارگذاری می‌کند و یک تابع برای تولید پاسخ بر اساس سؤال و متن زمینه تعریف می‌کند. سپس، مدل با یک سؤال و زمینه آزمایش شده و پاسخ تولیدشده چاپ می‌شود.

```
# Load the fine-tuned model
from google.colab import drive
drive.mount('/content/drive')

model_path = "/content/drive/MyDrive/fine-tuned-llama"

fine_tuned_model = AutoModelForCausalLM.from_pretrained(model_path)
fine_tuned_tokenizer = AutoTokenizer.from_pretrained(model_path)

# Define a function to generate answers
def generate_answer(question, context):
    input_text = f"question: {question} context: {context}"
    inputs = fine_tuned_tokenizer(input_text, return_tensors="pt")
    outputs = fine_tuned_model.generate(**inputs, max_length=512)
    answer = fine_tuned_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return answer

# Test the model with a new question and context
question = "پایتخت ایران کجاست؟"
context = "ایران کشوری در خاورمیانه است و پایتخت آن تهران می‌باشد."
answer = generate_answer(question, context)
print(f"Answer: {answer}")
```


این قطعه کد دو معیار ارزیابی مدل را محاسبه می‌کند: F1-Score (میانگین ماکرو) برای دقت متعادل و Exact Match برای درصد پیش‌بینی‌های کاملاً درست. سپس این معیارها برای یک مثال ساده محاسبه و چاپ می‌شوند.

```
[ ] from sklearn.metrics import f1_score

# Function to calculate F1-Score and Exact Match
def calculate_metrics(predictions, references):
    f1 = f1_score(references, predictions, average="macro")
    exact_match = sum([1 if p == r else 0 for p, r in zip(predictions, references)]) / len(references)
    return f1, exact_match

# Example usage
predictions = ["تهران", "پاریس"]
references = ["تهران", "تهران"]
f1, exact_match = calculate_metrics(predictions, references)
print(f"F1-Score: {f1}, Exact Match: {exact_match}")

F1-Score: 0.3333333333333333, Exact Match: 0.5

[ ] # Print final evaluation metrics
print(f"Final F1-Score: {f1}")
print(f"Final Exact Match: {exact_match}")

Final F1-Score: 0.3333333333333333
Final Exact Match: 0.5
```

ایجاد مدلی که از روش RAG برای بازیابی اطلاعات و پاسخ‌دهی به سوالات استفاده کند. برای بازیابی از BM25 یا TF-IDF و مدل زبانی از Llama-3.1-8B-4bit استفاده نمایید. برای مجموعه داده از فایل PDF پیوست شده امکان استفاده دارید. نتایج معیارهای F1-Score و Exact Match (EM) برای ارزیابی کیفیت مدل گزارش دهید.

در این قسمت چک می‌کنیم که فایل Drugs.pdf در گوگل درایو وجود داشته باشد.

```
[ ] from google.colab import drive
import os

# Mount Google Drive
drive.mount('/content/drive')

# Path to the PDF file in Google Drive
pdf_path = '/content/drive/My Drive/Drugs.pdf'

# Check if the file exists
if not os.path.exists(pdf_path):
    raise FileNotFoundError(f"The file {pdf_path} does not exist.")
```

در این قسمت میخواهیم از فایل متن رو استخراج کنیم و برای این کار از کتابخانه pyPDF2 استفاده می‌کنیم.

```
[ ] import PyPDF2

# Function to extract text from PDF
def extract_text_from_pdf(pdf_path):
    with open(pdf_path, 'rb') as file:
        reader = PyPDF2.PdfReader(file)
        text = ''
        for page in reader.pages:
            text += page.extract_text()
        return text

# Extract text from the PDF
text = extract_text_from_pdf(pdf_path)
```

در این بخش کارکترهای عددی و خاص حذف می‌شوند و تمام حروف‌ها به حروف کوچک تبدیل می‌شوند.

```
[ ] import re

# Function to preprocess text
def preprocess_text(text):
    # Remove special characters and digits
    text = re.sub(r'\W+', ' ', text)
    text = re.sub(r'\d+', '', text)
    # Convert to lowercase
    text = text.lower()
    return text

# Preprocess the text
processed_text = preprocess_text(text)
```

این قطعه کد متن پیش پردازش شده را به جملات و کلمات تقسیم می کند (توکن ایز می کند) و سپس دو مدل IDF TF- و BM25 را می سازد. این مدل ها برای تحلیل و رتبه بندی کلمات و جملات بر اساس اهمیت آنها در متن استفاده می شوند.

```
[ ] import nltk
    from nltk.corpus import stopwords # Import stopwords from NLTK

    # Download necessary NLTK data (if not already downloaded)
    nltk.download('punkt')
    nltk.download('stopwords')

    # Function to preprocess text
    def preprocess_text(text):
        # Remove special characters and digits
        text = re.sub(r'\W+', ' ', text)
        text = re.sub(r'\d+', '', text)
        # Convert to lowercase
        text = text.lower()
        return text

    # Preprocess the text
    processed_text = preprocess_text(text)

    # Tokenize the text
    tokenized_text = [nltk.word_tokenize(sentence) for sentence in processed_text.split('.')]

    # Build TF-IDF model
    from sklearn.feature_extraction.text import TfidfVectorizer

    # Use stopwords from NLTK
    tfidf_vectorizer = TfidfVectorizer(stop_words=stopwords.words('english'))
    tfidf_matrix = tfidf_vectorizer.fit_transform([' '.join(tokens) for tokens in tokenized_text])

    # Build BM25 model
    from rank_bm25 import BM25Okapi

    bm25 = BM25Okapi(tokenized_text)

    print("TF-IDF and BM25 models built successfully!")
```

این قطعه کد بررسی می کند که آیا CUDA (برای اجرای محاسبات روی GPU) در دسترس است یا خیر. همچنین نام دستگاه GPU نصب شده را چاپ می کند تا از فعال بودن آن اطمینان حاصل شود.

```
[ ] import nltk
    nltk.data.path.append('/root/nltk_data') # Add a writable directory

[ ] import torch
    print(torch.cuda.is_available())
    print(torch.cuda.get_device_name(0))
```

این قطعه کد یک مدل زبانی بزرگ (LLaMA-3-8B) را با استفاده از کتابخانه `transformers` بارگذاری می‌کند. مدل با تنظیمات کوانتیزاسیون ۴ بیتی (برای کاهش مصرف حافظه) و محاسبات با دقت `float16` روی GPU اجرا می‌شود.

```
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
import torch

model_name = "unsloth/llama-3-8b-bnb-4bit"
# Define the quantization configuration
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16
)

# Load the tokenizer and model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained([
    model_name,
    quantization_config=quantization_config,
    device_map="auto"
])

print("Model loaded successfully!")
```

این قطعه کد یک سیستم بازیابی اسناد را پیاده‌سازی می‌کند که از دو مدل BM25 و TF-IDF برای یافتن مرتبط‌ترین اسناد به یک کوئری استفاده می‌کند. نتایج از هر دو مدل ترکیب شده و اسناد برتر برمی‌گردند.

```
# Step 1: Retrieve documents with a smaller top_k
def retrieve_documents(query, top_k=3):
    # Preprocess the query
    processed_query = preprocess_text(query)
    tokenized_query = nltk.word_tokenize(processed_query)

    # Retrieve documents using BM25
    bm25_scores = bm25.get_scores(tokenized_query)
    top_indices = sorted(range(len(bm25_scores)), key=lambda i: bm25_scores[i], reverse=True)[:top_k]

    # Retrieve documents using TF-IDF
    query_vector = tfidf_vectorizer.transform([' '.join(tokenized_query)])
    tfidf_scores = (tfidf_matrix * query_vector.T).toarray().flatten()
    top_indices_tfidf = sorted(range(len(tfidf_scores)), key=lambda i: tfidf_scores[i], reverse=True)[:top_k]

    # Combine results from both models
    combined_indices = list(set(top_indices + top_indices_tfidf))
    return [tokenized_text[i] for i in combined_indices]
```

این قطعه کد با استفاده از مدل زبانی بارگذاری شده و اسناد بازیابی شده، یک پاسخ به سوال تولید می کند. ابتدا اسناد مرتبط به عنوان کانتکست ترکیب شده و سپس مدل زبانی با تنظیمات مشخص (مانند `max\_new\_tokens`) پاسخ را ایجاد می کند. در نهایت، پاسخ پیش بینی شده با پاسخ واقعی مقایسه می شود.

```
# Step 2: Generate answer with increased max_new_tokens and truncation
def generate_answer(query, retrieved_docs, max_length=512, max_new_tokens=200):
    # Combine retrieved documents into a single context
    context = ' '.join([' '.join(doc) for doc in retrieved_docs])

    # Truncate the context if it exceeds max_length
    inputs = tokenizer(
        f"Question: {query}\nContext: {context}\nAnswer:",
        return_tensors="pt",
        max_length=max_length,
        truncation=True
    )

    # Move input tensors to the same device as the model (GPU)
    inputs = {key: value.to(model.device) for key, value in inputs.items()}

    # Generate answer
    outputs = model.generate(**inputs, max_new_tokens=max_new_tokens)
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    return answer

# Step 3: Example usage
query = "What are the side effects of Drug X?"
true_answer = "The side effects of Drug X include nausea, headache, and dizziness."

retrieved_docs = retrieve_documents(query)
predicted_answer = generate_answer(query, retrieved_docs)

print(f"Predicted Answer: {predicted_answer}")
print(f"True Answer: {true_answer}")
```

این قطعه کد دو معیار ارزیابی F1-Score و Exact Match را برای مقایسه پاسخ پیش‌بینی‌شده با پاسخ واقعی محاسبه می‌کند. F1-Score همپوشانی توکن‌ها را اندازه‌گیری می‌کند، در حالی که Exact Match بررسی می‌کند که آیا پاسخ‌ها دقیقاً یکسان هستند یا خیر.

```
from sklearn.metrics import f1_score

def calculate_f1_score(predicted, true):
    predicted_tokens = set(nltk.word_tokenize(predicted.lower()))
    true_tokens = set(nltk.word_tokenize(true.lower()))
    common_tokens = predicted_tokens.intersection(true_tokens)
    if len(predicted_tokens) == 0 or len(true_tokens) == 0:
        return 0.0
    precision = len(common_tokens) / len(predicted_tokens)
    recall = len(common_tokens) / len(true_tokens)
    f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
    return f1

def calculate_exact_match(predicted, true):
    return 1 if predicted.lower().strip() == true.lower().strip() else 0

# Example evaluation
query = "What are the side effects of Drug X?"
true_answer = "The side effects of Drug X include nausea, headache, and dizziness."

retrieved_docs = retrieve_documents(query)
predicted_answer = generate_answer(query, retrieved_docs)

f1 = calculate_f1_score(predicted_answer, true_answer)
exact_match = calculate_exact_match(predicted_answer, true_answer)

print(f"F1-Score: {f1}")
print(f"Exact Match: {exact_match}")
```

این قطعه کد سیستم را روی چندین کوئری ارزیابی می‌کند. برای هر کوئری، اسناد مرتبط بازیابی شده و پاسخ پیش‌بینی‌شده تولید می‌شود. سپس F1-Score و Exact Match بین پاسخ پیش‌بینی‌شده و پاسخ واقعی محاسبه و نمایش داده می‌شوند. این فرآیند برای هر کوئری تکرار می‌شود.

```
# Example queries and true answers
queries = ["What is Drug X used for?", "What are the side effects of Drug Y?"]
true_answers = ["Drug X is used for treating condition A.", "The side effects of Drug Y include fatigue and rash."]

# Evaluate on multiple queries
for query, true_answer in zip(queries, true_answers):
    retrieved_docs = retrieve_documents(query)
    predicted_answer = generate_answer(query, retrieved_docs)

    f1 = calculate_f1_score(predicted_answer, true_answer)
    exact_match = calculate_exact_match(predicted_answer, true_answer)

    print(f"Query: {query}")
    print(f"Predicted Answer: {predicted_answer}")
    print(f"True Answer: {true_answer}")
    print(f"F1-Score: {f1}")
    print(f"Exact Match: {exact_match}")
    print("-" * 50)
```

در این قسمت f1-Score یک چیزی حدود 0.06 و exact match 0 نشان داده می‌شود.

این قطعه کد مدل و توکن‌ایزر آموزش‌دیده را در گوگل درایو ذخیره می‌کند و سپس آن‌ها را دوباره بارگذاری می‌کند. این کار برای استفاده‌های بعدی یا به‌اشتراک‌گذاری مدل مفید است.

```
# Save the model
model.save_pretrained("/content/drive/My Drive/llama_model")
tokenizer.save_pretrained("/content/drive/My Drive/llama_tokenizer")

# Load the model
model = AutoModelForCausalLM.from_pretrained("/content/drive/My Drive/llama_model")
tokenizer = AutoTokenizer.from_pretrained("/content/drive/My Drive/llama_tokenizer")
```

حالا برای بهتر شدن f1-score مدل زبانی خودمان را بر حسب زبان فارسی جلو می‌بریم.

این قطعه کد متن فارسی را پیش‌پردازش می‌کند. کاراکترهای فارسی نرمال‌سازی شده (مانند تبدیل "ی" به "ی" و "ك" به "ک")، علائم نگارشی و اعداد حذف شده و متن به حروف کوچک تبدیل می‌شود.

```
import re

# Function to preprocess Persian text
def preprocess_persian_text(text):
    # Normalize Persian characters (e.g., replace "ی" with "ی" and "ك" with "ک")
    text = text.replace("ي", "ی").replace("ك", "ک")

    # Remove punctuation and numbers
    text = re.sub(r'^\w\s', '', text)
    text = re.sub(r'\d+', '', text)

    # Convert to lowercase
    text = text.lower()

    return text

# Preprocess the text
processed_text = preprocess_persian_text(text)
```

این قطعه کد از کتابخانه `hazm` برای نرمال‌سازی و توکن‌ایز کردن متن فارسی استفاده می‌کند. متن نرمال‌سازی شده و سپس به جملات و کلمات تقسیم می‌شود.

```
[ ] from hazm import Normalizer, word_tokenize

# Normalize and tokenize Persian text
normalizer = Normalizer()
normalized_text = normalizer.normalize(processed_text)
tokenized_text = [word_tokenize(sentence) for sentence in normalized_text.split('. ')]
```

این قطعه کد دو مدل BM25 و TF-IDF را برای متن فارسی ایجاد می‌کند. BM25 برای بازیابی اسناد بر اساس شباهت به کوئری و TF-IDF برای وزندهی به کلمات بر اساس اهمیت آن‌ها در متن استفاده می‌شود.

```
from rank_bm25 import BM25Okapi
from sklearn.feature_extraction.text import TfidfVectorizer

# Build BM25 model
bm25 = BM25Okapi(tokenized_text)

# Build TF-IDF model
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform([' '.join(tokens) for tokens in tokenized_text])
```

این قطعه کد اسناد مرتبط با یک کوئری را با استفاده از دو مدل BM25 و TF-IDF بازیابی می‌کند. نتایج از هر دو مدل ترکیب شده و اسناد برتر برمی‌گردند.

```
def retrieve_documents(query, top_k=3):
    # Preprocess the query
    processed_query = preprocess_persian_text(query)
    tokenized_query = word_tokenize(processed_query)

    # Retrieve documents using BM25
    bm25_scores = bm25.get_scores(tokenized_query)
    top_indices = sorted(range(len(bm25_scores)), key=lambda i: bm25_scores[i], reverse=True)[:top_k]

    # Retrieve documents using TF-IDF
    query_vector = tfidf_vectorizer.transform([' '.join(tokenized_query)])
    tfidf_scores = (tfidf_matrix * query_vector.T).toarray().flatten()
    top_indices_tfidf = sorted(range(len(tfidf_scores)), key=lambda i: tfidf_scores[i], reverse=True)[:top_k]

    # Combine results from both models
    combined_indices = list(set(top_indices + top_indices_tfidf))
    return [tokenized_text[i] for i in combined_indices]
```


این قطعه کد با استفاده از مدل زبانی بارگذاری شده و اسناد بازیابی شده، یک پاسخ به سوال تولید می کند. ابتدا اسناد مرتبط به عنوان کانتکست ترکیب شده و سپس مدل زبانی با تنظیمات مشخص (مانند `max\_new\_tokens` و `num\_beams`) پاسخ را ایجاد می کند.

```
def generate_answer(query, retrieved_docs, max_length=512, max_new_tokens=200):
    # Combine retrieved documents into a single context
    context = ' '.join([' '.join(doc) for doc in retrieved_docs])

    # Prepare the input for the model
    input_text = f"Question: {query}\nContext: {context}\nAnswer:"

    # Tokenize the input and truncate if necessary
    inputs = tokenizer(
        input_text,
        return_tensors="pt",
        max_length=max_length,
        truncation=True
    )

    # Move input tensors to the same device as the model (GPU)
    inputs = {key: value.to(model.device) for key, value in inputs.items()}

    # Generate the answer
    outputs = model.generate(
        **inputs,
        max_new_tokens=max_new_tokens, # Control the length of the generated answer
        num_beams=5, # Use beam search for better quality
        temperature=0.7, # Lower temperature for more deterministic outputs
        early_stopping=True # Stop generation early if appropriate
    )

    # Decode the generated tokens to a string
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    return answer
```

این قطعه کد مدل Llama-3-8B را بارگذاری کرده و از آن برای تولید پاسخ به سوالات استفاده می‌کند. ابتدا اسناد بازیابی‌شده به عنوان کانتکست ترکیب شده و سپس مدل با تنظیمات مشخص (مانند `max\_length` و `max\_new\_tokens`) پاسخ را تولید می‌کند.

```
from transformers import AutoTokenizer, AutoModelForCausalLM

# Load Llama-3-8B model and tokenizer
model_name = "unsloth/llama-3-8b-bnb-4bit"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto")

def generate_answer(query, retrieved_docs, max_length=512, max_new_tokens=200):
    # Combine retrieved documents into a single context
    context = ' '.join([' '.join(doc) for doc in retrieved_docs])

    # Truncate the context if it exceeds max_length
    inputs = tokenizer(
        f"Question: {query}\nContext: {context}\nAnswer:",
        return_tensors="pt",
        max_length=max_length,
        truncation=True
    )

    # Move input tensors to the same device as the model (GPU)
    inputs = {key: value.to(model.device) for key, value in inputs.items()}

    # Generate answer
    outputs = model.generate(**inputs, max_new_tokens=max_new_tokens)
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    return answer
```

این قطعه کد سیستم را روی یک کوئری فارسی ارزیابی می‌کند. ابتدا اسناد مرتبط بازیابی شده و سپس پاسخ پیش‌بینی‌شده تولید می‌شود. در نهایت، F1-Score و Exact Match بین پاسخ پیش‌بینی‌شده و پاسخ واقعی محاسبه و نمایش داده می‌شوند.

```
from sklearn.metrics import f1_score

# Function to calculate F1 score
def calculate_f1_score(predicted, true):
    predicted_tokens = set(word_tokenize(predicted.lower()))
    true_tokens = set(word_tokenize(true.lower()))
    common_tokens = predicted_tokens.intersection(true_tokens)
    if len(predicted_tokens) == 0 or len(true_tokens) == 0:
        return 0.0
    precision = len(common_tokens) / len(predicted_tokens)
    recall = len(common_tokens) / len(true_tokens)
    f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
    return f1

# Function to calculate Exact Match
def calculate_exact_match(predicted, true):
    return 1 if predicted.lower().strip() == true.lower().strip() else 0

# Example evaluation
query = "چیست؟ X عوارض دارو" # Example query in Persian
true_answer = "شامل تهوع، سردرد و سرگیجه است X عوارض دارو" # Example true answer in Persian

retrieved_docs = retrieve_documents(query)
predicted_answer = generate_answer(query, retrieved_docs)

f1 = calculate_f1_score(predicted_answer, true_answer)
exact_match = calculate_exact_match(predicted_answer, true_answer)

print(f"F1-Score: {f1}")
print(f"Exact Match: {exact_match}")
```

در این قسمت f1-score و exact match به ترتیب برابر با 0.09 و 0 می‌آیند.

پس سعی میکنیم با انجام پیش پردازش مدل را ارتقا دهیم.

این قطعه کد از کتابخانه `hazm` برای حذف کلمات توقف (Stop Words) از متن فارسی استفاده می‌کند. کلمات توقف (مانند "و"، "در"، "با") که معمولاً در تحلیل‌های زبانی بی‌اهمیت هستند، از لیست توکن‌ها حذف می‌شوند.

```
[ ] from hazm import stopwords_list

# List of Persian stop words
persian_stopwords = set(stopwords_list())

def remove_stopwords(tokens):
    return [token for token in tokens if token not in persian_stopwords]

# Remove stop words from the tokenized text
tokenized_text = [remove_stopwords(tokens) for tokens in tokenized_text]
```

این قطعه کد از کتابخانه `hazm` برای ریشه‌یابی (Stemming) کلمات فارسی استفاده می‌کند. ریشه‌یابی کلمات را به شکل پایه‌ای خود تبدیل می‌کند تا تحلیل‌های بعدی ساده‌تر و دقیق‌تر انجام شود.

```
[ ] from hazm import Stemmer

# Initialize the Persian stemmer
stemmer = Stemmer()

def stem_tokens(tokens):
    return [stemmer.stem(token) for token in tokens]

# Stem the tokenized text
tokenized_text = [stem_tokens(tokens) for tokens in tokenized_text]
```

این قطعه کد دو مدل BM25 و TF-IDF را برای متن فارسی ایجاد می‌کند. BM25 برای بازیابی اسناد بر اساس شباهت به کوئری و TF-IDF برای وزن‌دهی به کلمات بر اساس اهمیت آن‌ها در متن استفاده می‌شود.

```
[ ] from rank_bm25 import BM25Okapi
    from sklearn.feature_extraction.text import TfidfVectorizer

# Build BM25 model
bm25 = BM25Okapi(tokenized_text)

# Build TF-IDF model
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform([' '.join(tokens) for tokens in tokenized_text])
```

این قطعه کد اسناد مرتبط با یک کوئری را با استفاده از دو مدل BM25 و TF-IDF بازیابی می‌کند. نتایج از هر دو مدل ترکیب شده و اسناد برتر برمی‌گردند.

```
[ ] def retrieve_documents(query, top_k=3):
    # Preprocess the query
    processed_query = preprocess_persian_text(query)
    tokenized_query = word_tokenize(processed_query)

    # Retrieve documents using BM25
    bm25_scores = bm25.get_scores(tokenized_query)
    top_indices = sorted(range(len(bm25_scores)), key=lambda i: bm25_scores[i], reverse=True)[:top_k]

    # Retrieve documents using TF-IDF
    query_vector = tfidf_vectorizer.transform([' '.join(tokenized_query)])
    tfidf_scores = (tfidf_matrix * query_vector.T).toarray().flatten()
    top_indices_tfidf = sorted(range(len(tfidf_scores)), key=lambda i: tfidf_scores[i], reverse=True)[:top_k]

    # Combine results from both models
    combined_indices = list(set(top_indices + top_indices_tfidf))
    return [tokenized_text[i] for i in combined_indices]
```

این قطعه کد با استفاده از مدل زبانی بارگذاری شده و اسناد بازیابی شده، یک پاسخ به سوال تولید می کند. ابتدا اسناد مرتبط به عنوان کانتکست ترکیب شده و سپس مدل زبانی با تنظیمات مشخص (مانند `max\_new\_tokens` و `num\_beams`) پاسخ را ایجاد می کند.

```
[ ] def generate_answer(query, retrieved_docs, max_length=512, max_new_tokens=200):
    # Combine retrieved documents into a single context
    context = ' '.join([' '.join(doc) for doc in retrieved_docs])

    # Prepare the input for the model
    input_text = f"Question: {query}\nContext: {context}\nAnswer:"

    # Tokenize the input and truncate if necessary
    inputs = tokenizer(
        input_text,
        return_tensors="pt",
        max_length=max_length,
        truncation=True
    )

    # Move input tensors to the same device as the model (GPU)
    inputs = {key: value.to(model.device) for key, value in inputs.items()}

    # Generate the answer
    outputs = model.generate(
        *inputs,
        max_new_tokens=max_new_tokens, # Control the length of the generated answer
        num_beams=5, # Use beam search for better quality
        temperature=0.7, # Lower temperature for more deterministic outputs
        early_stopping=True # Stop generation early if appropriate
    )

    # Decode the generated tokens to a string
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    return answer
```

این قطعه کد مدل Llama-3-8B را بارگذاری کرده و از آن برای تولید پاسخ به سوالات فارسی استفاده می‌کند. ابتدا دستگاه (GPU یا CPU) تشخیص داده شده و مدل و داده‌ها به آن منتقل می‌شوند. سپس پاسخ با تنظیمات مشخص (مانند `max\_new\_tokens`) تولید و نمایش داده می‌شود.

```
[ ] import torch
    from transformers import AutoTokenizer, AutoModelForCausalLM

    # Check if GPU is available
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"Using device: {device}")

    # Load the model
    model_name = "unsloth/llama-3-8b-bnb-4bit"
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForCausalLM.from_pretrained(model_name, device_map=device) # Use GPU or CPU

    # Move model to GPU (if available)
    model = model.to(device)

    # Example input
    input_text = "چیست؟ X عوارض دارو"
    inputs = tokenizer(input_text, return_tensors="pt")

    # Move input tensors to GPU (if available)
    inputs = {key: value.to(device) for key, value in inputs.items()}

    # Generate output
    outputs = model.generate(**inputs, max_new_tokens=100)
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    print(f"Generated Answer: {answer}")
```

این قطعه کد دو معیار ارزیابی برای پاسخ‌های تولید شده توسط یک سیستم پاسخ‌دهی (مثلاً یک چت‌بات یا سیستم بازیابی اطلاعات) محاسبه می‌کند :

1. F1-Score: دقت و یادآوری (Precision) و (Recall) را بر اساس تعداد توکن‌های مشترک بین پاسخ پیش‌بینی شده و پاسخ واقعی محاسبه کرده و F1-Score را به عنوان میانگین هارمونیک این دو مقدار برمی‌گرداند .

2. Exact Match: بررسی می‌کند آیا پاسخ پیش‌بینی شده دقیقاً با پاسخ واقعی یکسان است یا خیر (با نادیده گرفتن فاصله‌ها و حروف بزرگ/کوچک).

در نهایت، این دو معیار برای یک مثال خاص (با استفاده از توابع `retrieve\_documents` و `generate\_answer`) محاسبه و چاپ می‌شوند.

```

from sklearn.metrics import f1_score

# Function to calculate F1 score
def calculate_f1_score(predicted, true):
    predicted_tokens = set(word_tokenize(predicted.lower()))
    true_tokens = set(word_tokenize(true.lower()))
    common_tokens = predicted_tokens.intersection(true_tokens)
    if len(predicted_tokens) == 0 or len(true_tokens) == 0:
        return 0.0
    precision = len(common_tokens) / len(predicted_tokens)
    recall = len(common_tokens) / len(true_tokens)
    f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
    return f1

# Function to calculate Exact Match
def calculate_exact_match(predicted, true):
    return 1 if predicted.lower().strip() == true.lower().strip() else 0

# Example evaluation
query = "چیست؟ X عوارض دارو" # Example query in Persian
true_answer = "شامل تهوع، سردرد و سرگیجه است. X عوارض دارو" # Example true answer in Persian

retrieved_docs = retrieve_documents(query)
predicted_answer = generate_answer(query, retrieved_docs)

f1 = calculate_f1_score(predicted_answer, true_answer)
exact_match = calculate_exact_match(predicted_answer, true_answer)

print(f"F1-Score: {f1}")
print(f"Exact Match: {exact_match}")

```

بعد از این قسمت f1-score، 0.05 و exact match، 0 میشوند. که یعنی پیش‌پردازش‌ها با ما کمکی نکرده‌اند.